Markets in a Spin

Adeesh Devasthale Mrittika Giri Nishant Lamboria
Sauparna Kar Siddhartha Bhattacharya

Indian Statistical Institute, Kolkata

May 15, 2026

What is the Idea?

- Financial markets consist of many **interacting assets**
- **Objective:**
  - Model **cross-sectional dependence**
  - Capture **temporal changes** in market structure
- **Approach:**
  - A statistical physics inspired model which involves **interacting particle systems**

The Ising Model

- Consider n particles, each taking state $+1$ or -1.
- The particle states are random and generally **dependent** .
- Proposed by Lenz and Ising (1920s) to model interacting systems.
- In its generalized form,

$$P(\sigma) = \frac{1}{Z(W, h)} \exp\left(\sum_{i<j} w_{ij}\sigma_i\sigma_j + \sum_i h_i\sigma_i\right)$$

- Here:
  - $W = ((w_{ij}))$: symmetric interaction matrix
  - $h = (h_i)$: individual bias vector
  - $Z(W, h)$: normalizing constant (partition function)

Market as a Configuration

- Consider a selection of n stocks
- For each day t, define:

$$\boldsymbol{X}_t = (X_{1,t}, X_{2,t}, \dots, X_{n,t}) \in \{-1, +1\}^n$$

- Each stock is represented as:

$$X_{i,t} = \begin{cases} +1 & \text{if price of the } i^{\text{th}} \text{ stock goes up on day } t \\ -1 & \text{if price of the } i^{\text{th}} \text{ stock goes down on day } t \end{cases}$$

- Indexed over time,

$$\boldsymbol{X}_{t \geq 0} \quad \text{forms a stochastic process}$$

A Naive Approach

- Assume a single interaction structure for the market
- Model:

$$P(\boldsymbol{X}_t = x) \propto \exp\left(\frac{1}{2}x^\top W x + h^\top x\right)$$

- **The Obvious Limitations:**
  - The structure remains static across time.
  - There is a lack of temporal-covariance structure (i.e. joint distribution)

The Proposed Solution

- Introduce time-varying parameters (W_t, h_t)
- Model:

$$P(\boldsymbol{X}_t = x \mid (W_t, h_t)) \propto \exp\left(\frac{1}{2}x^\top W_t x + h_t^\top x\right)$$

  where $(W_t, h_t)_{t \geq 0}$ evolves as a stochastic process over time
- Mitigates the static–nature problem
- Captures temporal dependence structure

The Problems in the Proposed Solution

- **Computational difficulty of the model!**
  - The partition function is intractable:

$$Z(W_t, h_t) = \sum_{x \in \{-1,+1\}^n} \exp\left(\frac{1}{2} x^\top W_t x + h_t^\top x\right)$$

- Instead of specifying the evolution of (W_t, h_t), we focus on a more direct and tractable approach.

A Dynamic Interaction Model

Modelling Markets via Spin Models

Conditional Modeling Approach

- Modify the distribution of $\boldsymbol{X}_t$ as

$$P(\boldsymbol{X}_t = x \mid \boldsymbol{X}_{t-1}) \propto \exp\left(x^\top W \boldsymbol{X}_{t-1} + h^\top x\right)$$

- **Intuition** : Interaction effects appear with a time lag
- Implication: Conditional on $\boldsymbol{X}_{t-1}$, the components of $\boldsymbol{X}_t$ become mutually independent!

Logistic Structure of the Conditional Model

- For each stock i at time t,

$$P(X_{i,t} = 1 \mid \boldsymbol{X}_{t-1}) = \left[1 + \exp\left(-2 \left(\sum_{\substack{j=1 \\ j \neq i}}^{n} W_{ij}(t)\, X_{j,t-1} + h_i \right) \right) \right]^{-1}$$

- Taking logits gives a linear form:

$$\operatorname{logit}(P(X_{i,t} = 1 \mid \boldsymbol{X}_{t-1})) = 2 \left(\sum_{\substack{j=1 \\ j \neq i}}^{n} W_{ij}(t)\, X_{j,t-1} + h_i \right)$$

- **NOTE:** $W_{ii}(t) = 0 \ \forall \ i \implies$ only cross-asset dependencies are estimated.

Rolling Window Estimation

- Parameters are estimated on a rolling window of fixed length L

$$\mathcal{W}_t = \{t - L + 1, \dots, t\}$$

- **Assumptions:**
  - approximate local stationarity within each window
  - gradual temporal evolution of the interaction network

- Estimate:

$$W(t) = \big(W_{ij}(t)\big)_{1 \le i,j \le n}, \quad h(t) = \big(h_1(t), \dots, h_n(t)\big)^\top$$

Conditional Likelihood

- For a fixed window $\mathcal{W}_t$, define

$$\eta_{i,s}(t) = h_i(t) + \sum_{j \neq i} W_{ij}(t)\, X_{j,s-1}.$$

- The likelihood is

$$L_t(W(t), h(t)) = \prod_{s=t-L+1}^{t} \prod_{i=1}^{n} \frac{\exp\big(X_{i,s}\, \eta_{i,s}(t)\big)}{2\cosh\big(\eta_{i,s}(t)\big)}.$$

- Hence the log-likelihood becomes

$$\ell_t(W(t), h(t)) = \sum_{s=t-L+1}^{t} \sum_{i=1}^{n} \Big[X_{i,s}\, \eta_{i,s}(t) - \log\big(2\cosh(\eta_{i,s}(t))\big)\Big].$$

ℓ_1-Regularized Estimation

- The number of interaction parameters grows quadratically with n
- Impose an ℓ_1-penalty:

$$\lambda \|W_i(t)\|_1 = \lambda \sum_{j \neq i} |W_{ij}(t)|$$

- Penalized objective:

$$(\hat{W}(t), \hat{h}(t)) = \underset{W(t),\, h(t)}{\arg\max} \left\{ \ell_t(W(t), h(t)) - \lambda \sum_{i=1}^{n} \|W_i(t)\|_1 \right\}$$

- Since $\hat{W}(t)$ need not be symmetric, we symmetrise it via

$$\hat{W}_{\mathrm{sym}}(t) = \frac{\hat{W}(t) + \hat{W}(t)^\top}{2}$$

Coordinate Descent

- The ℓ_1-penalty encourages sparsity, so weak interactions are shrunk to zero.
- The optimization is convex, but the ℓ_1-term makes it non-smooth.
- We therefore update the coefficients one at a time, holding the others fixed.
- This is done iteratively:
  - update one entry of $W(t)$,
  - move to the next coefficient,
  - repeat until the objective stabilizes.
- The effect of the penalty is a soft-thresholding behavior: small coefficients are pushed to zero, while stronger interactions are retained.
- This yields a sparse and interpretable interaction network for each rolling window.

Prediction

- Given the observed market configuration $\boldsymbol{X}_t$,

$$\hat{P}(X_{i,t+1} = 1 \mid \boldsymbol{X}_t) = \left[1 + \exp(-2\,(\hat{\eta}_{i,t+1}(t)))\right]^{-1}$$

where:

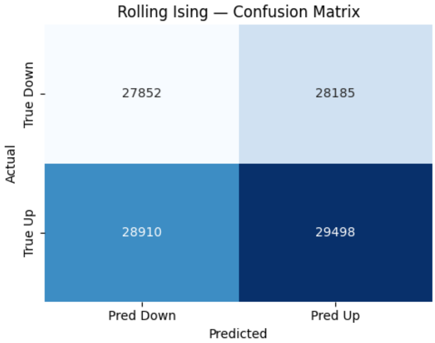
$$\hat{\eta}_{i,t+1}(t) = \hat{h}_i(t) + \sum_{j \neq i} \hat{W}_{ij}(t)\, X_{j,t}$$

- The prediction rule is

$$\hat{X}_{i,t+1} = \begin{cases} +1, & \text{if } \hat{\eta}_{i,t+1}(t) \geq 0, \\ -1, & \text{otherwise.} \end{cases}$$

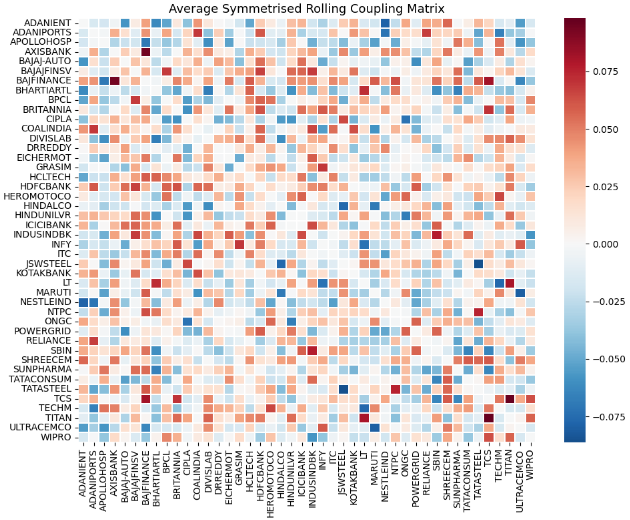
Empirical Results

- We evaluate the model on the latest Nifty50 constituents.
- Sample period: 17/11/2017 to 01/01/2025.
- Rolling window length: 30 trading days.
- **Performance:**
  - Accuracy: 50.50% (+1) and 49.70% (-1)
  - Precision = 0.5114
  - Recall = 0.5050
  - F1 Score = 0.5082



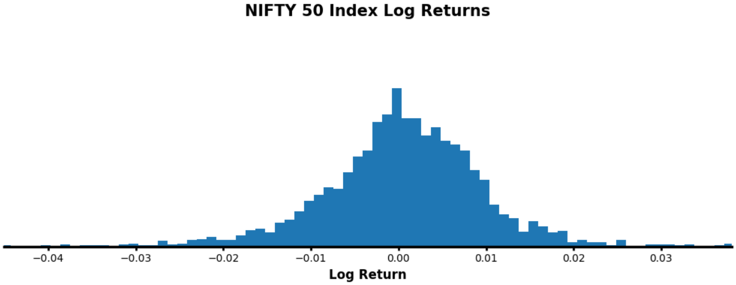
Confusion matrix

Sample Estimated Interaction Matrix



Can we do better?

- The binary model forces every day into two states: $X_{i,t} \in \{-1, +1\}$
- In the data, returns are often clustered near zero.
- Treating these near zero movements as either up or down can introduce noise.



An Alternative?

- To avoid treating weak movements as meaningful directional changes, we introduce a **neutral state**.
- Thus, each stock is modeled as

$$X_{i,t} \in \{-1, 0, +1\},$$

where
  - $+1$: positive movement,
  - 0: neutral / low-activity movement,
  - -1: negative movement.
- This gives a better representation of market behaviour and naturally leads to the **Blume–Capel model**.

Beyond Binary Spins

Modelling Markets via Spin Models

Blume–Capel Interaction Model

- Let
$$X_t = (X_{1,t}, \dots, X_{n,t}), \qquad X_{i,t} \in \{-1, 0, +1\}.$$
- The market configuration at time t is modeled through the Hamiltonian
$$H(X_t;\theta) = -\sum_{i=1}^{n} h_i X_{i,t} - \sum_{i=1}^{n} d_i X_{i,t}^2 - \sum_{1\le i<j\le n} J_{ij} X_{i,t} X_{j,t}.$$
- The corresponding probability distribution is
$$P(X_t = x) = \frac{1}{Z(\theta)} \exp\big(-H(x;\theta)\big),$$
where
$$Z(\theta) = \sum_{x\in\{-1,0,+1\}^n} \exp\big(-H(x;\theta)\big)$$
is the partition function.

Adaptive PCA Preprocessing

- Directly modeling interactions between all N assets leads to a very high-dimensional system.
- To obtain a lower-dimensional latent representation, we apply **Principal Component Analysis (PCA)** .
- Let
$$r_t \in \mathbb{R}^N$$
denote the cross-sectional return vector at time t.
- At each walk-forward step, we use a training window of length W:
$$\mathcal{W}_t = \{r_{t-W+1}, \ldots, r_t\}.$$
- Organize these returns into the data matrix
$$R_t \in \mathbb{R}^{W \times N},$$
whose rows correspond to the observations in $\mathcal{W}_t$.

Principal Component Scores

- First center the data:

$$\mu_t = \frac{1}{W} R_t^\top \mathbf{1}_W, \qquad \widetilde{R}_t = R_t - \mathbf{1}_W \mu_t^\top .$$

- Compute the singular value decomposition:

$$\widetilde{R}_t = U \Sigma V^\top .$$

- Let

$$P_t = V_{:,1:n}$$

denote the matrix of the leading n principal directions.

- The latent principal component scores are obtained by projection:

$$z_t = P_t^\top (r_t - \mu_t) \in \mathbb{R}^n .$$

Three-State Spin Encoding

- The Blume–Capel model requires discrete spins, so each PC score is mapped into

$$s_t \in \{-1, 0, +1\}^n.$$

- For the j-th PC component, let

$$\{z_{1,j}, \ldots, z_{W,j}\}$$

  be the training scores.
- Given a quantile parameter $q \in (0, 1/3]$, define

$$\tau_j^{\mathrm{lo}} = Q_q(\{z_{t,j}\}), \qquad \tau_j^{\mathrm{hi}} = Q_{1-q}(\{z_{t,j}\}),$$

- The spin encoding is defined as

$$s_j = \begin{cases} -1 & \text{if } z_j < \tau_j^{\mathrm{lo}}, \\ 0 & \text{if } \tau_j^{\mathrm{lo}} \le z_j \le \tau_j^{\mathrm{hi}}, \\ +1 & \text{if } z_j > \tau_j^{\mathrm{hi}}. \end{cases}$$

Why Exact Inference is Hard

- The latent spin configuration $X_t \in \{-1, 0, +1\}^n$ is modeled by a Blume–Capel distribution:

$$p(X_t = x \mid \theta) = \frac{1}{Z(\theta)} \exp\big(-H(x;\theta)\big).$$

- The normalizing constant is

$$Z(\theta) = \sum_{x \in \{-1,0,+1\}^n} \exp\big(-H(x;\theta)\big).$$

- Since the state space has size 3^n, exact computation is infeasible even for moderate n.
- This makes the exact likelihood, exact marginals, and exact parameter learning computationally intractable.

Mean-Field Approximation

- To make inference tractable, we replace the full joint distribution by a product approximation:

$$q(x) = \prod_{i=1}^{n} q_i(x_i).$$

- Each node is then influenced by an **effective field** rather than the exact configuration of all other nodes:

$$h_i^{\text{eff}} = h_i + \sum_{j \neq i} J_{ij} m_j, \qquad m_j = \mathbb{E}_q[X_j].$$

- Under this approximation, the one-site probabilities become

$$q_i(+1) \propto e^{d_i + h_i^{\text{eff}}}, \qquad q_i(0) \propto 1, \qquad q_i(-1) \propto e^{d_i - h_i^{\text{eff}}}.$$

Mean-Field Magnetization and Activity

- From the one-site marginals, we define the local magnetization

$$m_i = \mathbb{E}_q[X_i] = q_i(+1) - q_i(-1),$$

  and the local activity

$$q_i^{(2)} = \mathbb{E}_q[X_i^2] = q_i(+1) + q_i(-1).$$

- These quantities satisfy a self-consistent fixed-point relation, since

$$h_i^{\text{eff}} = h_i + \sum_{j \neq i} J_{ij} m_j.$$

- **Interpretation** :
  - m_i measures directional bias,
  - $q_i^{(2)}$ measures activity away from the neutral state.

Pseudo-Likelihood Estimation

- We still need a way to estimate the parameters $\theta = (h, d, J)$.
- Since direct likelihood maximization is hard, we use the pseudo-likelihood:

$$\mathrm{PLL}(x;\theta) = \sum_{i=1}^{n} \log p(x_i \mid x_{-i};\theta).$$

- For each node i, the conditional depends on the local field

$$h_i^{\mathrm{eff}}(x_{-i}) = h_i + \sum_{j\neq i} J_{ij} x_j.$$

- Thus the parameters can be updated from tractable local conditionals rather than from the full joint law.

Why Both Approximations Are Needed

- The Blume–Capel model presents two separate computational challenges:
  - **Inference Problem:** exact evaluation of

$$Z(\theta) = \sum_{x \in \{-1,0,+1\}^n} e^{-H(x;\theta)}$$

    is computationally intractable.
  - **Learning Problem:** parameter estimation through the full likelihood is infeasible.

- Therefore:
  - **Mean-field approximation** replaces the exact partition function by a tractable approximation $Z_{\mathrm{MF}}(\theta)$.
  - **Pseudo-likelihood estimation** replaces the full likelihood by node-wise conditional probabilities.

Behaviour of the Market

- Stable markets are expected to show weak interactions, while crises are expected to induce strong collective behavior.
- We classify the market into three latent regimes: bullish, bearish, and neutral.
- A **Hidden Markov Model** (HMM) is used to identify these hidden regimes.

Mixture Model

- Let $C_t \in \{1, 2, \ldots, K\}$ denote the latent market regime at time t.
- Conditional on the regime $C_t = k$, the latent spin configuration

$$X_t \in \{-1, 0, +1\}^n$$

is assumed to follow a regime-specific Blume–Capel distribution:

$$X_t \mid (C_t = k) \sim p_{\mathrm{BC}}(\,\cdot\,; \theta^{(k)}),$$

where

$$\theta^{(k)} = \left(h^{(k)}, d^{(k)}, J^{(k)}\right).$$

- Marginally, X_t follows a mixture of regime-specific Blume–Capel models:

$$p(x_t) = \sum_{k=1}^{K} P(C_t = k)\, p_{\mathrm{BC}}(x_t; \theta^{(k)}).$$

Hidden Markov Model Notations

- We assume that $\{C_t\}$ is the Markov Chain of the hidden regimes.
- The **state space** of C_t is
$$\mathcal{C} = \{1, 2, 3, \ldots, K\}.$$
- **Transition Probabilities:**
$$A_{ij} := P(c_t = j \mid c_{t-1} = i).$$
- **Emission model:**
$$e_{t,k} := p(x_t \mid c_t = k) \approx p_{MF}(x_t \mid \theta^{(k)}).$$

Forward Equations

- Define the forward variable

$$\alpha_{t,k} := P(C_t = k \mid X_{1:t}).$$

- The forward recursion is given by

$$\widetilde{\alpha}_{t,k} = \sum_{i=1}^{K} \alpha_{t-1,i} A_{ik},$$

$$\alpha_{t,k} \propto p_{\mathrm{MF}}(X_t \mid \theta^{(k)}) \widetilde{\alpha}_{t,k}.$$

- After normalization, α_t forms a probability vector.

Transition Equations

- **Posterior Transition Probability**

$$\xi_{t,ij} := P(C_{t-1} = i,\ C_t = j \mid X_{1:t}).$$

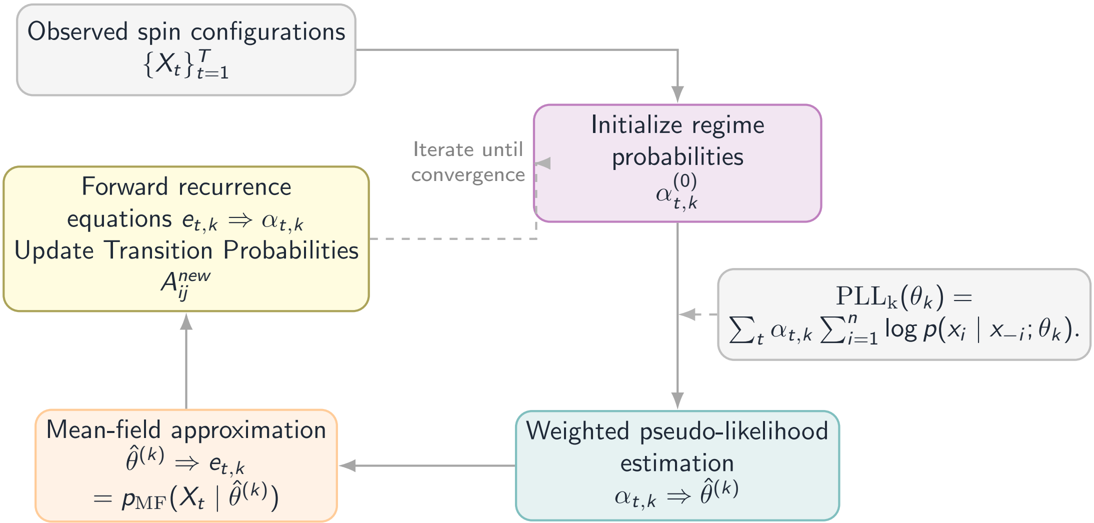
- **Posterior Transition Probability Update**

$$\xi_{t,ij} = \frac{\alpha_{t-1,i}\, A_{ij}\, e_{t,j}}{\sum_k \alpha_{t-1,i}\, A_{ik}\, e_{t,k}}$$

- **Transition Probability Update**

$$A_{ij}^{\text{new}} = \frac{\sum_{t=1}^{T} \xi_{t,i,j}}{\sum_{t=1}^{T} \sum_{k=1}^{K} \xi_{t,i,k}}.$$

Modified EM Algorithm



From Signals to Strategies

Modelling Markets via Spin Models

Online Parameter Learning: Weighted Objective

- At each time step, the HMM provides soft regime responsibilities

$$w_{t,k} = \alpha_{t,k}$$

- **Weighted PLL objective** for regime k:

$$\mathcal{L}_t^{(k)}(\boldsymbol{\theta}^{(k)}) = w_{t,k} \cdot \mathrm{PLL}(\boldsymbol{x}_t; \boldsymbol{\theta}^{(k)}) - \Omega(\boldsymbol{\theta}^{(k)})$$

- $\Omega(\cdot)$ is a **composite regularisation** functional that ensures strong convexity and sparse coupling

Composite Regularisation Functional

$$\Omega(\boldsymbol{\theta}) = \frac{\lambda_2}{2}\|\boldsymbol{h}\|_2^2 + \frac{\lambda_2^{(d)}}{2}\|\boldsymbol{d}\|_2^2 + \frac{\lambda_2}{2}\|\boldsymbol{J}\|_F^2 + \lambda_1^{(J)}\|\boldsymbol{J}\|_{1,\text{off-diag}}$$

- λ_2 : L2 on $\boldsymbol{h}, \boldsymbol{J}$ — prevents overfitting
- $\lambda_2^{(d)}$: L2 on $\boldsymbol{d}$ — prevents gradient saturation
- $\lambda_1^{(J)}$: L1 on off-diagonal J_{ij} — induces sparse coupling structure

Online Mirror Descent with AdaGrad

- Maintain diagonal preconditioner $\mathbf{G}_t$:

$$\mathbf{G}_t = \mathbf{G}_{t-1} + \mathrm{diag}(\boldsymbol{g}_t^{\odot 2}), \qquad \mathbf{G}_0 = \mathbf{I}$$

- **AdaGrad update**:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta\,(\mathbf{G}_t^{1/2} + \varepsilon\mathbf{I})^{-1}\boldsymbol{g}_t$$

- $\boldsymbol{g}_t = \nabla_{\boldsymbol{\theta}}\mathcal{L}_t$ is the stochastic gradient
- $\varepsilon = 10^{-8}$ for numerical stability

OMD Update: Algorithm in Practice

- **Gradient step** (weighted negative PLL + L2):

$$\tilde{\boldsymbol{g}}_h = -w\, \boldsymbol{g}_h + \lambda_2 \boldsymbol{h}$$
$$\tilde{\boldsymbol{g}}_d = -w\, \boldsymbol{g}_d + \lambda_2^{(d)} \boldsymbol{d}$$
$$\tilde{\boldsymbol{g}}_J = -w\, \boldsymbol{g}_J + \lambda_2 \boldsymbol{J}$$

- **Post-update projections**:
  - **Soft-threshold $\boldsymbol{J}$** for ℓ_1 sparsity (Proximal Gradient Descent):

$$J'_{ij} = \mathrm{sign}(J_{ij}) \max\left(|J_{ij}| - \lambda_1^{(J)},\, 0\right)$$

  - **Clip** $(\boldsymbol{h}, \boldsymbol{d}, \boldsymbol{J})$ to bounded ranges.
  - **Symmetrise** and **project** $\boldsymbol{J}$:

$$\boldsymbol{J}' \leftarrow \frac{\boldsymbol{J}' + \boldsymbol{J}'^{\top}}{2}, \qquad \mathrm{diag}(\boldsymbol{J}') \leftarrow \boldsymbol{0}$$

Transition Matrix Update

- The HMM transition matrix **A** is updated by **exponentiated gradient** (EG) descent
- Expected transition count:

$$\xi_{ij} = \frac{\alpha_{t-1,i}\, A_{ij}\, e_{t,j}}{\sum_k \alpha_{t-1,i}\, A_{ik}\, e_{t,k}}$$

- Update in log-space for stability:

$$L_{ij} = \log(A_{ij} + 10^{-300}) - \eta_A\, g_{ij}, \qquad g_{ij} = -\frac{\xi_{ij}}{A_{ij} + 10^{-300}}$$

- Row-wise softmax normalisation ensures **A** stays stochastic

Blume–Capel Signal Forecast

- Given current HMM belief $\boldsymbol{\alpha}_t$ and regime magnetisations $\{\boldsymbol{m}^{(k)}\}$:
- **H-step ahead expected spins**:

$$\bar{\boldsymbol{s}}_{t+H} = \sum_{k=1}^{K} \alpha_{t+H,k} \cdot \boldsymbol{m}^{(k)}$$

- Regime forecast: $\boldsymbol{\alpha}_{t+H} = \mathbf{A}^H \boldsymbol{\alpha}_t$
- Map back to asset space via inverse PCA:

$$\boldsymbol{s}_t^{\mathrm{BC}} = \mathbf{P}\, \bar{\boldsymbol{s}}_{t+H} \in \mathbb{R}^N$$

Momentum Signal

- Multi-window momentum for lookbacks $\mathcal{W} = (w_1, w_2, w_3)$:

$$m_t^{(i)}(w) = \frac{1}{w} \sum_{\tau=0}^{w-1} r_{t-\tau}^{(i)}$$

- **Cross-sectional standardisation**:

$$\hat{m}_t^{(i)}(w) = \frac{m_t^{(i)}(w) - \mathrm{mean}_j\, m_t^{(j)}(w)}{\mathrm{std}_j\, m_t^{(j)}(w) + 10^{-8}}$$

- Combined momentum signal:

$$\boldsymbol{s}_t^{\mathrm{mom}} = \operatorname*{mean}_{w \in \mathcal{W}} \hat{\boldsymbol{m}}_t(w)$$

Adaptive Blender

- Maintain rolling windows of historical **Information Coefficients** (Spearman ρ_S)
- **Blend weights** from IC averages:

$$w_{\mathrm{BC}} = \max\left\{ \frac{1}{W_{\mathrm{IC}}} \sum_{\tau \in \mathcal{I}_{\mathrm{BC}}} \rho_{S,\tau},\ w_{\min} \right\}$$

- Normalised weights:

$$\tilde{w}_{\mathrm{BC}} = \frac{w_{\mathrm{BC}}}{w_{\mathrm{BC}} + w_{\mathrm{mom}}}, \qquad \tilde{w}_{\mathrm{mom}} = \frac{w_{\mathrm{mom}}}{w_{\mathrm{BC}} + w_{\mathrm{mom}}}$$

- If insufficient history (< 10 obs), fall back to base weights

Regime Gate

- Track **per-regime directional accuracy** in deque $\mathcal{H}_k$:

$$\mathrm{acc}_k = \frac{1}{|\mathcal{H}_k|} \sum_{h \in \mathcal{H}_k} h$$

- **Gate scaling**:

$$g_k = \begin{cases} 1.0 & \text{if } \mathrm{acc}_k \geq \mathrm{acc}_{\text{min}} \text{ or } |\mathcal{H}_k| < W_{\mathrm{warmup}} \\ \max\{g_{\text{min}}, 0.35\} & \text{otherwise} \end{cases}$$

- Final signal:

$$\boldsymbol{s}_t^{\mathrm{final}} = g_{c_t} \cdot \boldsymbol{s}_t^{\mathrm{blended}}$$

- Down-scales the model when a regime is performing poorly

Signal-to-Weight Mapping

- Raw weights depend on trading mode:

Mode	Formula
Signal	$w_i = s_i \cdot \mathbf{1}_{[\lvert s_i \rvert > \theta_{\mathrm{entry}}]}$
Equal	$w_i = \dfrac{\mathrm{sign}(s_i)}{\sum_j \mathbf{1}_{[\mathrm{sign}(s_j)=\mathrm{sign}(s_i)]}}$
Long-Only	$w_i = \max\{s_i, 0\} \cdot \mathbf{1}_{[\lvert s_i \rvert > \theta_{\mathrm{entry}}]}$
Vol-Scaled	$w_i = \dfrac{s_i}{\sigma_i + 10^{-8}} \cdot \mathbf{1}_{[\lvert s_i \rvert > \theta_{\mathrm{entry}}]}$

Risk Management Layers

- **Layer 1 – Short scaling**:

$$w_i \leftarrow w_i \cdot \gamma_{\text{short}} \quad \text{for } w_i < 0$$

(default $\gamma_{\text{short}} = 0$ for long-only)

- **Layer 2 – Gross normalisation**:

$$\boldsymbol{w} \leftarrow \frac{\boldsymbol{w}}{\|\boldsymbol{w}\|_1} \quad (\text{if } \|\boldsymbol{w}\|_1 > 0)$$

- **Layer 3 – Per-asset cap**:

$$w_i \leftarrow \text{clip}(w_i, -w_{\max}, +w_{\max})$$

- **Layer 4 – Re-normalisation** to unit gross exposure

Volatility Scaling

- Annualised realised volatility from strategy P&L:

$$\sigma_{\mathrm{ann}} = \mathsf{std}(\{r_{\mathrm{pnl},\tau}\}) \cdot \sqrt{252}$$

- **Leverage scalar**:

$$L_t = \mathsf{clip}\left(\frac{\sigma_{\mathrm{target}}}{\sigma_{\mathrm{ann}} + 10^{-6}},\ 0.1,\ L_{\max}\right)$$

- Default: $\sigma_{\mathrm{target}} = 15\%$ annualised, $L_{\max} = 2.0$
- **Regime confidence scaling** (optional):

$$C_t = c_{\min} + (1 - c_{\min}) \cdot \frac{\max_k \alpha_{t,k} - 1/K}{1 - 1/K + 10^{-8}}$$

Transaction Costs and P&L Attribution

- **Turnover** at rebalancing:
$$\mathrm{TO}_t = \|\boldsymbol{w}_t - \boldsymbol{w}_{t-1}\|_1$$

- **Cost drag** (with cost rate c in decimal):
$$\mathrm{Cost}_t = c \cdot \mathrm{TO}_t$$

- **Daily P&L** for H-day holding period:
$$\mathrm{PnL}_{t+d} = \left\langle \frac{\boldsymbol{w}_t}{H}, \boldsymbol{r}_{t+d} \right\rangle - \mathrm{Cost}_t \cdot \mathbf{1}_{[d=1]}, \qquad d = 1, \ldots, H$$

True Walk-Forward Validation

- **No peeking**:
  - PCA, spin thresholds, and model parameters use only past data
- **Incremental learning**:
  - Between refits, HMM filter updates beliefs but parameters are frozen
- **No in-sample bias**:
  - Evaluation is always on **future returns** not seen during training
- Satisfies the requirement for **true walk-forward validation** with incremental online learning

The Three Control Parameters

- **Forecast Horizon** H : how many days ahead we predict and hold
- **Refit Frequency** F : how often the full model is retrained from scratch
- **Training Window** W : history used for each refit

Parameter	Value Chosen	Role
H	5 days	Prediction + holding period
F	5 days	PCA + thresholds + reset
W	126 days	6 months of history for SVD

Three Interleaved Time Scales

- **Refit events** (every F days): full retrain at $t = 130, 135, 140, \ldots$
- **Prediction horizons** (every F days, length H): forecast at t for $t + 1 : t + H$
- **Online updates** (every day): incremental SGD on $\boldsymbol{\theta}^{(k)}$ and $\mathbf{A}$

Phase 1: Refit at Day $t = 130$

- Training window: $\mathcal{W}_{130} = \{\boldsymbol{r}_5, \ldots, \boldsymbol{r}_{130}\}$ ($W = 126$ days)
- **What is computed:**
  1. PCA: $(\boldsymbol{\mu}_{130}, \mathbf{P}_{130}) \in \mathbb{R}^{50} \times \mathbb{R}^{50 \times n_{\mathrm{PCA}}}$
  2. Spin thresholds: $(\tau_j^{\mathrm{lo}}, \tau_j^{\mathrm{hi}})_{j=1}^{5}$ via z-score at $q = 0.33$
  3. Encode history: $\boldsymbol{x}_\tau = \mathrm{encode}(\mathbf{P}_{130}^\top(\boldsymbol{r}_\tau - \boldsymbol{\mu}_{130}))$
  4. Train RARSI (Algorithm 6): initialise $\boldsymbol{\theta}^{(k)}$, $\mathbf{A}$, AdaGrad states
- **Frozen until day 135:** $\boldsymbol{\mu}_{130}$, $\mathbf{P}_{130}$, spin thresholds
- Free to evolve: $\boldsymbol{\theta}^{(k)}$, $\mathbf{A}$, $\boldsymbol{\alpha}_t$, AdaGrad accumulators

Phase 2: Online Operation (Days 131–135)

At each day t, observe $\boldsymbol{r}_t$ and execute:

1. **Encode:** $\boldsymbol{x}_t = \text{encode}(\mathbf{P}_{130}^{\top}(\boldsymbol{r}_t - \boldsymbol{\mu}_{130}))$ PCA frozen
2. **Emit:** $e_{t,k} = p_{\mathrm{MF}}(\boldsymbol{x}_t \mid \boldsymbol{\theta}^{(k)})$
3. **Filter:** $\boldsymbol{\alpha}_t = \text{HMMFilter.step}(\boldsymbol{e}_t; \mathbf{A}, \boldsymbol{\alpha}_{t-1})$ **$\boldsymbol{\alpha}_t$ updated**
4. **Weight:** $w_{t,k} = \max(\alpha_{t,k}, 0.01) / \sum_k \max(\alpha_{t,k}, 0.01)$
5. **OMD:** $\boldsymbol{\theta}^{(k)} \leftarrow \text{OMD.step}(\boldsymbol{\theta}^{(k)}, \boldsymbol{x}_t, w_{t,k})$ **$\boldsymbol{\theta}^{(k)}$ updated**
6. **EG:** $\mathbf{A} \leftarrow \text{TransitionEG.step}(\mathbf{A}, \boldsymbol{\alpha}_{t-1}, \boldsymbol{e}_t)$ **$\mathbf{A}$ updated**

No trading yet, this is the warmup/learning phase after refit.

Phase 3: Prediction at Day $t = 135$

Generate forecast for days 136–140 ($H = 5$):

Step	Computation
1	Regime forecast: $\boldsymbol{\alpha}_{140} = \mathbf{A}^5 \boldsymbol{\alpha}_{135}$
2	Magnetisations: $\boldsymbol{m}^{(k)}$ from current $\boldsymbol{\theta}^{(k)}$
3	Expected spins: $\bar{\boldsymbol{s}}_{140} = \sum_k \alpha_{140,k}\, \boldsymbol{m}^{(k)} \in (-1, +1)^{n_{\mathrm{PCA}}}$
4	Asset signal: $\boldsymbol{s}_{135}^{\mathrm{BC}} = \mathbf{P}_{130}\, \bar{\boldsymbol{s}}_{140} \in \mathbb{R}^{50}$
5	Momentum: $\boldsymbol{s}_{135}^{\mathrm{mom}}$ from raw returns
6	Blend: $\boldsymbol{s}_{135}^{\mathrm{blended}} = \tilde{w}_{\mathrm{BC}} \boldsymbol{s}_{135}^{\mathrm{BC}} + \tilde{w}_{\mathrm{mom}} \boldsymbol{s}_{135}^{\mathrm{mom}}$
7	Gate: $\boldsymbol{s}_{135}^{\mathrm{final}} = g_{c_{135}} \cdot \boldsymbol{s}_{135}^{\mathrm{blended}}$
8	Portfolio: $\boldsymbol{w}_{135} \leftarrow \mathrm{PortfolioEngine}(\boldsymbol{s}_{135}^{\mathrm{final}})$

Weights $\boldsymbol{w}_{135}$ are **held unchanged** for days 136–140.

Phase 4 & 5: Refit Timing and Execution

- **Morning of day 135:** generate forecast using old PCA $(\boldsymbol{\mu}_{130}, \mathbf{P}_{130})$ and evolved $\boldsymbol{\theta}^{(k)}$
- **Evening of day 135:** observe $\boldsymbol{r}_{135}$; refit PCA on $\mathcal{W}_{135} = \{\boldsymbol{r}_{10}, \ldots, \boldsymbol{r}_{135}\}$
- **Overnight:** compute new $(\boldsymbol{\mu}_{135}, \mathbf{P}_{135})$, retrain RARSI from scratch
- **Day 136 onwards:** new model structure is live; online updates resume

Holding period P&L:

$$r_{\mathrm{pnl},\,135+d} = \left\langle \frac{\boldsymbol{w}_{135}}{H}, \boldsymbol{r}_{135+d} \right\rangle - \mathrm{Cost}_{135} \cdot \mathbf{1}_{[d=1]}, \qquad d = 1, \ldots, 5$$

What is Fixed vs. What is Dynamic

Component	Fixed For	Update Rule	Frequency
$(\boldsymbol{\mu}, \mathbf{P})$ PCA	$F = 5$ days	Full SVD recompute	Refit only
Spin thresholds	$F = 5$ days	Z-score on train scores	Refit only
$\theta^{(k)} = (\boldsymbol{h}, \boldsymbol{d}, \mathbf{J})$	—	OMD (AdaGrad)	Every day
$\mathbf{A}$ transitions	—	Exponentiated gradient	Every day
$\boldsymbol{\alpha}_t$ beliefs	—	Forward filter	Every day
AdaGrad $\mathbf{G}$ states	—	Accumulate $\boldsymbol{g}^{\odot 2}$	Every day

The F-day cycle gives **structural stability** (PCA, thresholds) while the daily online loop gives adaptive responsiveness (parameters, beliefs).

Key Insight: The Hybrid Design

- **Why not refit everything daily?**
  - PCA is expensive ($\mathcal{O}(\min(W^2N, WN^2))$)
  - Spin thresholds need statistical stability — daily jitter creates noise
  - Full retraining discards accumulated AdaGrad curvature
- **Why not freeze everything for F days?**
  - Blume–Capel parameters $\boldsymbol{\theta}^{(k)}$ must track slowly-changing market structure
  - HMM beliefs $\boldsymbol{\alpha}_t$ must react to new observations immediately
  - Without daily updates, the model becomes stale

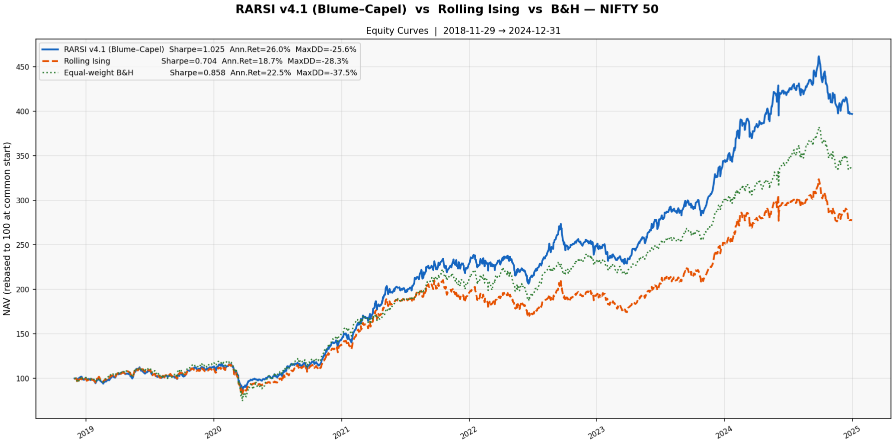
Results & Conclusions

Modelling Markets via Spin Models

Performance Comparison

Metric	Brief Definition	Blume Capel	Rolling Ising	Equal-weight B&H
Annualized Return	Compounded yearly portfolio return	26.02%	18.67%	22.54%
Annualized Volatility	Standard deviation of yearly returns (risk)	17.79%	17.22%	18.04%
Sharpe Ratio	Return earned per unit of total risk	1.025	0.704	0.858
Sortino Ratio	Return earned per unit of downside risk	0.084	0.054	0.063
Maximum Drawdown	Largest peak-to-trough portfolio decline	−25.62%	−28.32%	−37.54%
Calmar Ratio	Annual return divided by maximum drawdown	1.028	0.661	0.607
Hit Rate	Percentage of profitable trading periods	56.62%	56.42%	57.75%

NAV Graph



RARSI v4.1 (Blume-Capel) vs Rolling Ising vs B&H — NIFTY 50
Equity Curves | 2018-11-29 → 2024-12-31
RARSI v4.1 (Blume-Capel) Sharpe=1.025 Ann.Ret=26.0% MaxDD=-25.6%
Rolling Ising Sharpe=0.704 Ann.Ret=18.7% MaxDD=-28.3%
Equal-weight B&H Sharpe=0.858 Ann.Ret=22.5% MaxDD=-37.5%
NAV (rebased to 100 at common start)
450
400
350
300
250
200
150
100
2019
2020
2021
2022
2023
2024
2025

Conclusions: Modeling Insights

- Static interaction structures were unable to produce reliable predictive performance in a non-stationary market environment.
- Binary spin representations discarded a large amount of low-volatility market information, leading to weak precision and recall.
- Introducing a neutral state produced a more stable latent representation and improved signal quality.
- The Blume–Capel framework successfully captured:
  - directional dependence through J_{ij},
  - activity and inactivity through the crystal field d_i.

Conclusions: Inference and Performance

- Mean-field approximation and pseudo-likelihood estimation made online inference computationally feasible despite the exponential state space.
- The hidden Markov framework allowed the model to adapt to changing market regimes dynamically.
- Online learning and adaptive parameter updates improved robustness under non-stationary market conditions.
- The regime-aware tri-state model achieved significantly stronger predictive and portfolio performance relative to benchmark strategies such as the NIFTY50.

Thank You

Questions & Discussion